

```
1 #这节课，教会我们如何评估神经网络的参数是否健康，以及如何调整参数的统计学特征，使得它变得健康
2 # =====一些总结=====
3 # (1) 一层训练的基本过程：
4 #     1.我们规定一个神经网络层是这样的：
5 #         假设整体神经网络，只有一个训练样本输入-----比如当前文件的神经网络，只输入一个元组来训练
6 #         一个层就是从上一层获得上一层的激活值，到输出自己的激活值的过程
7 #     2.获得上一层激活值。激活值是一个 $1 \times n$ 的矩阵。当前层有 $m$ 个神经元
8 #     3.把上一层的激活值，和当前 $w$ 权重矩阵相乘。
9 #         权重矩阵是 $n \times m$ 的。
10 #         因为我们希望，上一层的每一个激活值，都会作为下一层 $m$ 个神经元中的每一个神经元的输入的构成之一
11 #         所以上一层的每一个激活值，都会和下一层的每一个神经元有一个连接edge。edge上面有一个唯一的权重 $w$ 
12 #         所以 $n$ 个激活值， $m$ 个当前层神经元-----共需要 $n \times m$ 个权重
13 #     4.和 $w$ 矩阵乘完了之后，获得 $1 \times m$ 的矩阵
14 #     5.然后这每一个元素，逐个输入tanh等激活函数，获得输出的 $1 \times m$ 的向量
15 #     6.输出的 $1 \times m$ 向量，就是当前层的激活值
16 #
17 #
18
19 # (2) 我们对每一层进行的归一化，是不是为了保证 $w$ 可以更加健康地更新？也就是每次学习的更新率不大也不小。
20
21
22
23
24 # (3) 本处神经网络的层级结构：
25 #     embedding层 --> 第一线性层 --> BN层 --> tanh层 --> 第二线性层 --> logits
26
27
28
29 # (4) 过拟合和欠拟合
30
31
32
33
34
35 #=====导入基本的库=====
36 import torch
37 import torch.nn.functional as F
```

```
38 import matplotlib.pyplot as plt # for making figures
39
40 #=====初始化数据集=====
41 #完成读入, 并且构造字典
42
43 words = open('Neural Network_Zero to hero/makemore/names.txt', 'r').read().splitlines()
44
45
46 chars = sorted(list(set(''.join(words))))
47 stoi = {s:i+1 for i,s in enumerate(chars)}
48 stoi['.'] = 0
49 itos = {i:s for s,i in stoi.items()}
50 vocab_size = len(itos)
51 print(itos)
52 print(vocab_size)
53
54
55
56 #=====数据集分类, 以及定义context长度=====
57 block_size = 3 # 也就是上下文长度: 读入多少个上文, 来预测下一个char
58 def build_dataset(words):
59     X, Y = [], []
60
61     for w in words:
62         context = [0] * block_size
63         for ch in w + '.':
64             ix = stoi[ch]
65             X.append(context)
66             Y.append(ix)
67             context = context[1:] + [ix] # crop and append
68
69     X = torch.tensor(X)
70     Y = torch.tensor(Y)
71     print(X.shape, Y.shape)
72     return X, Y
73
74 import random
```

```

75 random.seed(42)
76 random.shuffle(words)
77 n1 = int(0.8*len(words))
78 n2 = int(0.9*len(words))
79
80 Xtr, Ytr = build_dataset(words[:n1])      # 80%
81 Xdev, Ydev = build_dataset(words[n1:n2])  # 10%
82 Xte, Yte = build_dataset(words[n2:])      # 10%
83
84
85 #=====初始化MLP=====
86 def simple_MLP(n_embd=10, n_hidden=200, max_steps=200000, batch_size=32):
87     # 回忆一下, n_embd表示的是, 一个字符对应的向量维度
88     g = torch.Generator().manual_seed(2147483647)
89     #指定随机数种子, 方便我们复现结果, 因为同一个种子生成的随机数都一样
90
91     # (1) 初始化后的参数, 存在这样一个问题:
92     #     1.理论上来说, 没经过任何学习的模型, 输出27个字符的概率应该是相同的。
93     #         也就是logits内的元素取值差别不大, 所以最终计算出的Loss是3.2958左右
94     #     2.但是, 我们初始化的参数是随机的, 很可能导致初次输出的27个字符的概率差别很大, 首次的Loss会很大
95     #     3.为什么这是个问题?
96     #         1.loss 大 → 梯度大 → 更新步长大。第一步更新就可能把参数推到垃圾区域, 甚至出现 nan
97     #         2.浪费训练步数
98     #         3.这是 Karpathy 想强调的核心。训练前先跑一遍评估, 如果初始 loss ≈ 3.2958, 那么数据管道大概率是正确的
99     #         如果一上来 loss 就是 27, 你就分不清是"初始化尺度不好"还是"数据写错了"。
100    #         一旦修好初始化让它稳定在 3.29, 这个数字就变成了一个免费的、极其灵敏的 bug 探测器。
101
102    # 4.如何解决这个问题?
103    #     1.注意到Logit是由隐藏层输出H@W2+B2计算的,
104    #         所以我们希望W2的元素尽可能小, B2偏置初始值为0
105    #         这样算出来的logits彼此之间数值相差就不会太大
106    #
107    # 5.第四点提出的解决方法是不是有问题:
108    #     问题:
109    #         1.logits = H @ W2 + B2是计算最终输出的矩阵运算式子。H表示激活函数tanh的输出
110    #         2.既然W2整体经过*0.01的过程, 来使得所有的值更接近0。然后再乘以H
111    #         3.那么宏观来看, 是不是可以等效于给每一个H里面的分量, 乘0.01? -----是

```

```

112 #         4.如果是，那么H里面的每一个分量，相对大小不是压根没改变吗？-----是
113 #         5.计算交叉熵的公式：先计算H每一个分量出现概率，然后取log，然后相加，然后再整体取负号-----否
114 #         6.这样算出来的结果，应该是没区别的-----否
115 #
116 #     回忆Loss是如何计算得到的：
117 #         1.softmax计算式子我理解错了。我得重新理解softmax在干什么。
118 #         2.最终得到的logits应该是这样的规格：(batch_size,vocab_size)
119 #             batch_size表示当前一次训练，使用了多少个训练集。vocab_size表示下一个ch的可能取值有27个
120 #             我们取batch_size = 32, vocab_size = 27
121 #         3.对应地，我们得到的yb应该是(32,1)。表示对于32个输入，我们都有一个正确的输出
122 #         4.我们这里使用交叉熵来计算softmax：
123 #             loss = - (1/N) *  $\sum_i \log( \exp(\text{logits}[i, Yb[i]]) / \sum_j \exp(\text{logits}[i, j]) )$ 
124 #         5. $\exp(\text{logits}[i, Yb[i]]) / \sum_j \exp(\text{logits}[i, j])$ :
125 #             表示计算，第i个样本作为神经网络的输入，神经网络输出正确ch的概率。
126 #         6.然后对于每一个样本我们都计算输出正确ch的概率，然后分别取log，再取负
127 #         7.然后关于每一个样本，来求平均值
128 #
129 #     为什么这样可以使得Loss减小：
130 #         看式子：loss = - (1/N) *  $\sum_i \log( \exp(\text{logits}[i, Yb[i]]) / \sum_j \exp(\text{logits}[i, j]) )$ 
131 #         W2乘0.01，宏观上确实近似于，给logits乘0.01.带入式子看一下就知道相当于给Loss乘一个怎么样的系数了
132 #
133 #
134 #
135 # (2)中间的隐藏层的激活函数tanh的处理，在反向传播的时候会出现“死神经元”问题，或者说过饱和问题：
136 #     1.我们关注隐藏层的输出H矩阵是怎么来的：tanh(embed@W1+B1)
137 #     2.我们发现，输出矩阵H的很多值，都直接是1和-1。即，hpreact的很多值离零太远，只能被映射到及其接近±1
138 #     3.所以对H这个Tensor矩阵内的每个元素进行backpropagate：
139 #         1.每一个元素都有父元素和子元素。假设当前元素为A
140 #         2.父元素就是所有对A当前data值有贡献的元素，子元素就是用A元素data作为输入计算自己data值的元素
141 #         3.这里H矩阵是(32,200)，因为一次训练输入32组元组，隐藏层有200个神经元
142 #         4.那么32行中的一行里面，每一个元素就对应就是一个神经元的输出
143 #         5.所以Hpreact就是经过tanh处理之前的H。并且元素之间是一一对应的关系
144 #         6.反向传播，就是接受子元素的grad，计算自己的grad，并且把自己的grad传给父类
145 #     4.所以当我们尝试对H的元素求backward的时候，发现：
146 #         1.H中元素，有的接近或者干脆就浮点溢出变成了±1
147 #         2.对这样的元素backward，传给它的父元素的grad就几乎是0
148 #         3.所以当在一个隐藏层中的神经元，对于所有可能输入的元组，取值都是正负一

```

```
149 #         那么当它反向传播的时候，传给它父元素的都几乎是0，不论是什么元组
150 #         4. 于是就出现了一个“死神经元”
151
152
153
154 # (3) 我们也可以直接尝试对hpreact矩阵处理
155 #     1.hpreact矩阵是由embedding@W1+B1得到的
156 #     2.我们可以通过减小W1和B1的值，使得hpreact内的元素更接近0，从而tanh之后映射到的值域离±1远一点
157
158
159 # (4) Andrej提到，他希望：most of the Neural Net to have relatively similar activation
160 #     1.那么什么是activation?
161 #         # 1. 激活 (activation) 就是神经网络中某一层的“输出值”。
162 #         # 2. 它是该层接收输入，完成线性变换（如矩阵乘加偏置），再经过激活函数（如tanh、ReLU）后得到的结果。
163 #         # 3. 在你的代码里，h = torch.tanh(hpreact) 中的 h 就是这一层的激活值。
164 #         # 4. 之所以叫“激活”，是因为它模仿了生物神经元：数值越大，表示这个神经元“越兴奋”（输出越强）。
165
166 #     2.Andrej这句话更具体的意思：
167 #         1.是希望每一层的输出值分布（均值、方差）都比较接近。
168 #         2.其实我觉得，他的意思应该是，希望每一层神经层：
169 #         接收到的激活值，经过w权重矩阵处理之后的当前层预激活值，以及最终输出的激活值，数值上的分布都近似
170 #
171
172 #     3.为什么这样希望？
173 #         1. 反向传播的本质是链式乘法。
174 #         损失 L 对某一层参数的梯度，等于从输出层回传到该层的多个局部导数连乘。
175 #         如果每一层的激活值分布差异巨大，这些局部导数也会差异巨大。
176 #         连乘之后，梯度可能指数级放大（爆炸）或指数级缩小（消失）。
177 #
178 #         2. 激活值分布直接影响激活函数的局部导数。
179 #         以 tanh 为例：
180 #         当输入很大（如 5 或 -5），tanh 输出接近 1 或 -1，导数接近 0。对它反向传播，父元素难以更新grad
181 #         当输入很小（如 0.01），tanh 近似线性，导数接近 1。
182 #         如果某层激活值太大，进入饱和区，该层梯度几乎为 0，参数无法更新。
183 #         如果某层激活值太小，信号微弱，学习速度极慢。
184 #
185 #         3. 如果各层激活分布相似（比如均值 0，方差约 1）。
```

```
186 #         那么每一层的输入都落在激活函数的“健康区间”：
187 #         不是完全线性（表达能力弱），也不是完全饱和（梯度消失）。
188 #         这样每一层都能获得大小适中的梯度，参数更新稳定。
189 #
190 #     4. 分布逐层剧烈变化会导致训练失衡。
191 #         假设第一层输出方差 1，第二层方差 100，第三层方差 0.001。
192 #         反向传播时，第三层的梯度会被放大或缩小，传到第一层时已经面目全非。
193 #         结果是：某些层学得飞快，某些层几乎不动，整体难以收敛。
194 #
195 #     5. 相似分布还能避免“内部协变量偏移”的恶化。
196 #         每一层的输入分布如果随训练不断剧烈变化，
197 #         后层需要不断适应前层输出的新分布，训练效率低。
198 #         保持分布相对稳定，后层可以更专注地学习真正的模式。
199 #
200 #
201 #     4. 分布近似如何有利于控制数值范围？
202 #         我感觉控制每一层激活值都是标准正态分布，是有助于控制数值范围。但是控制数值范围也不一定保持标准正态分布吧？
203 #         只是说，大致地维护在标准正态分布的数值，性质很优秀。比如极端值少并且大部分值都在0周围。
204 #         但是优秀的数值分布不全是标准正态分布。只是工程上把激活值维护成标准正态分布很好
205 #
206 #
207 #     5. 前面提到，预激活值（也就是tanh等激活函数的输入）太大不好，那么（预）激活值太小又怎么样？
208 #         1. 如果预激活值都在 0 附近（比如 -0.01 到 0.01），tanh 在这个区间几乎就是一条直线：
209 #             那么这一层等于没做非线性变换。
210 #             多层这样的线性层叠加，数学上等价于一层线性层。
211 #             网络表达能力退化成单层，学不到复杂模式。
212 #
213 #         2. 如果激活值很小，可能使得下一层梯度更新慢：
214 #             假设我们的网络层次是这样的：
215 #              $X$ （上一层激活值） $\rightarrow$  线性层 $W1$ （得到预激活值  $z$ ） $\rightarrow$   $\tanh$ （得到激活值  $a$ ） $\rightarrow$  线性层 $W2$ （得到  $y$ ） $\rightarrow$  损失  $L$ 
216 #             简单来说， $L$ 可以如此表达： $L = W2 * \tanh(X * W1)$ 
217 #             那么我在计算 $dL/dW1$ 的时候：
218 #             根据chain rule，我们有： $dL/dW1 = (dL/dz) * (dz/dW1)$ 
219 #             并且 $Z = X * W1$ ，所以 $dz/dW1$ 就是 $x$ ，也就是上一层的激活值。
220 #             那么如果 $x$ 很小，那么 $dL/dW1$ 的值就非常小——所以 $W1$ 梯度就很小，反向传播的时候，传给 $x$ 的grad就小。使得 $x$ 更新慢
221 #
222 #
```

```
223 # (5) 工程上，一种普适的对参数进行修正的方法：
224 #     1. 对每一层的W矩阵乘上这个项：gain / sqrt(fan_in)。
225 #     2. fan_in是每一层的输入向量的维度。也就是上一层的输出数量。
226 #     3. gain是一个修正因子——根据我们每一层的激活函数决定。
227 #     至于为什么这样的修正可以保持，当前层的获取到的上一层的激活值，和当前层输出的激活值，在数值分布上相似，就看notes的数学原理
228 #     因为内容太多，解释起来很复杂
229
230
231
232 # (6) 第五点提到的初始化方法的优势和局限：
233 #     1. 它能保证初始状态下的数值分布：
234 #         通过 1/sqrt(fan_in) 缩放，抵消了输入维度对输出方差的影响。
235 #         通过 gain 补偿激活函数的方差压缩。
236 #         最终使每一层线性层的输出（激活值）方差大致保持在 1 左右。
237 #         这就保证了网络在刚开始训练时，每一层的数值分布都是健康的。
238 #
239
240 #     2. 但它不是万能的，有几个重要前提和局限：
241 #         前提一：假设输入数据是独立同分布，且均值为 0、方差为 1。
242 #         如果输入数据本身分布很奇怪（比如均值很大、方差极小），初始化效果会打折扣。
243 #
244 #         前提二：它只保证初始时的分布。
245 #         训练一旦开始，权重会更新，分布会随之变化。
246 #         它不能保证训练过程中的每一步，各层激活值都保持完美分布。
247 #
248 #         前提三：它主要针对线性层。
249 #         对于卷积层、循环层、注意力层等更复杂的结构，方差传播的数学公式不同，
250 #         需要各自的初始化策略（如 Transformer 有自己的特殊缩放）。
251 #
252 #         前提四：它无法解决“训练后期”的分布漂移问题。
253 #         随着训练进行，权重可能变得很大或很小，激活值分布仍可能恶化。
254 #
255
256 #     3. 为了弥补这些局限，现代深度学习还引入了其他技术：
257 #         - 归一化层（BatchNorm、LayerNorm）：在训练过程中动态强制激活值分布稳定。
258 #         - 残差连接（ResNet）：让梯度绕过某些层，缓解深层网络的方差漂移。
259 #         - 更精细的初始化（如 LSUV、Fixup）：进一步优化初始方差。
```

```

260 # - 学习率预热 (warmup) : 训练初期用较小学习率, 避免初始权重被破坏。
261 #
262 C = torch.randn((vocab_size, n_embd), generator=g)
263 W1 = torch.randn((n_embd * block_size, n_hidden), generator=g)*5/3*(block_size*n_embd**0.5)
264 b1 = torch.randn(n_hidden, generator=g)*0.01#
265 #----batch_norm层实际上使得这个b1失效了。加上和没加一样。这里就留着吧, 提醒自己
266 W2 = torch.randn((n_hidden, vocab_size), generator=g)*0.01
267 b2 = torch.randn(vocab_size, generator=g)*0
268 # (7) 上面初始化的各项参数含义:
269 # vocab_size是, 训练集内部字符种类。比如我们的名字训练集, 这个数据就是26个小写字母+一个空格符
270 # n_embd表示: 一个ch, 要使用多长的向量来表示
271 # block_size就是context有多少个字符
272
273
274 # (8) 初始化并且获取BatchNorm层参数
275 batch_gain, batch_bias, batch_mean_running, batch_std_running = get_batchNorm_parameters(n_hidden)
276
277
278 parameters = [C, W1, b1, W2, b2, batch_gain, batch_bias]
279 print(sum(p.nelement() for p in parameters)) # number of parameters in total
280 for p in parameters:
281     p.requires_grad = True
282
283
284 # same optimization as last time
285 lossi = []
286
287 for i in range(max_steps):
288
289     #每次随机选择小样本来进行训练。大小就是batch_size
290     ix = torch.randint(0, Xtr.shape[0], (batch_size,), generator=g)
291     Xb, Yb = Xtr[ix], Ytr[ix] # batch X,Y
292
293     # forward pass
294     emb = C[Xb] # embed the characters into vectors
295     embcat = emb.view(emb.shape[0], -1) # concatenate the vectors
296     hpreact = embcat @ W1 + b1 # hidden layer pre-activation

```

```
297
298     #run BatchNorm layer-----我们总体层级架构是: embd层, 线性层, BN层, tanh层, 第二线性层
299     hpreact = run_batch_norm_layer(hpreact, batch_gain, batch_bias, batch_mean_running, batch_std_running)
300
301
302     h = torch.tanh(hpreact) # hidden layer
303     logits = h @ W2 + b2 # output layer
304     loss = F.cross_entropy(logits, Yb) # loss function
305
306     # backward pass
307     for p in parameters:
308         p.grad = None
309     loss.backward()
310
311     # update
312     lr = 0.1 if i < 100000 else 0.01 # step learning rate decay
313     for p in parameters:
314         p.data += -lr * p.grad
315
316     # track stats
317     if i % 10000 == 0: # print every once in a while
318         print(f'{i:7d}/{max_steps:7d}: {loss.item():.4f}')
319         lossi.append(loss.log10().item())
320     # 把训练好的参数交出去, 供 split_loss 使用
321     return C, W1, b1, W2, b2, lossi
322
323
324 #=====在每个数据划分 (split) 上, 计算模型的损失值。=====
325 #计算Loss需要参数们, 还有数据集作为输入
326 #split就是一个字符串, 告诉该函数, 要选择哪个分区来计算Loss
327 @torch.no_grad() # this decorator disables gradient tracking
328 def split_loss(split, C, W1, b1, W2, b2):
329     x,y = {
330         'train': (Xtr, Ytr),
331         'val': (Xdev, Ydev),
332         'test': (Xte, Yte),
333     }[split]
```

```
334     emb = C[x] # (N, block_size, n_embd)
335     embcat = emb.view(emb.shape[0], -1) # concat into (N, block_size * n_embd)
336     h = torch.tanh(embcat @ W1 + b1) # (N, n_hidden)
337     logits = h @ W2 + b2 # (N, vocab_size)
338     loss = F.cross_entropy(logits, y)
339     print(split, loss.item())
340     return loss.item()
341
342
343
344 #=====batch_normal=====
345 #BatchNorm 不是必须，但它解决一个具体问题：
346 # 线性层输出 hpreact = x @ W + b 的数值分布会随训练变。
347 # 太大，tanh 饱和，梯度接近 0；太小，又学得慢。只靠初始化，很难一直保持合适
348 # 我们就希望在隐藏层的线性层之后、激活函数之前，加一个BatchNormal层，动态的维护预激活值，使得它的数值性质保持的很好
349 # 这一BatchNorm层的参数是根据当前样本Batch计算而来的
350
351 # (0) 看下面的函数，理解BatchNorm实际在做什么？为什么这样可以重新归一化？
352 def get_batchNorm_parameters(n_hidden):
353     batch_gain = torch.ones((1, n_hidden))
354     batch_bias = torch.zeros((1, n_hidden))
355     batch_mean_running = torch.zeros((1, n_hidden))
356     batch_std_running = torch.ones((1, n_hidden))
357     return batch_gain, batch_bias, batch_mean_running, batch_std_running
358
359 def run_batch_norm_layer(hpreact, batch_gain, batch_bias, batch_mean_running, batch_std_running, training=True):
360 # (1) 这一步操作为什么可以完成归一化？
361 # 1.hpreact.mean(0, keepdim=True)表示跨过第0维求平均。
362 # 比如这里的hpreact规格是(32, 100)。也就是32行，100列。
363 # 跨过每一行，沿着列求平均。最后得到的结果是 (1, 100) 。也就是32个样本，关于100个分量中的每一个都求一次平均
364 # 2.hpreact.std(0, keepdim=True)同理。跨过行求std标准差
365 # 3.至于为什么可以完成归一化。。。我们仔细观察式子，可以很轻松发现，这个式子符合概统里面标准化随机变量的格式
366 # 本质是对每一个hpreact元素进行平移和放缩
367
368
369 # (2) 那么更改一下格式，全部变成沿着第1维，是不是可以有同样的效果？
370 # 1.不一样。沿着第0维：
```

```
371 #         平均对象是：每个特征，跨所有样本
372 #         对于同一特征，不同样本减同一个值。而这个值是根据不同样本的同一个位置的特征值计算的
373 #         所以不同样本的信息是耦合的
374 #     2. 沿着第一维度：
375 #         平均对象是：每个样本，跨所有特征
376 #         同一样本，不同特征减同一个值
377 #         样本间彼此独立，不互相影响。因为加减的值只来自本样本
378
379
380 # (3) batch_gain, batch_bias是如何被使用的？
381 #     1. batch_gain是一个(1,n_hidden)的矩阵，hpreact是一个(batch_size,n_hidden)的矩阵，两者相乘不是矩阵乘法：
382 #         而是Pytorch自带的逐元素乘法。通过广播机制。Batch_gain会被拓展成(batch_size,n_hidden)的矩阵，每一行都是一样的向量
383 #         然后乘法的规则就是：hpreact[i, j] * batch_gain[i, j]。实际上就是hpreact[i, j] * batch_gain[0, j]
384 #     2. 所以就相当于，hpreact的第j列的元素，都乘了batch_gain[0, j]，也就是Batch_size个样本值，的第j个特征分量，都乘了同一个数
385 #     3. 同理，加上batch_bias就相当于：
386 #         给hpreact的第j列的元素，都加上了batch_bias[0, j]。也就是Batch_size个样本值，的第j个特征分量，都加了同一个数
387
388
389 # (4) 我们初始化的时候，batch_gain都是1，batch_bias都是0，对hpreact矩阵数值没影响。那这两个gain和bias矩阵有什么意义？
390 #     1. 归一化强制把每个特征变成零均值、单位方差。这只是一个假设，不一定对所有特征都好。
391 #     2. 可能对于某些特征参数而言，使得Loss最低的方向应该是均值为0.5，于是加上bias就允许它偏移
392 #     3. 也可能我们希望特征参数偏学到饱和的非线性。
393 #     4. 这两个参数矩阵，是可以被backward遍历到的。是可以被更新的。神经网络可以通过这两个参数矩阵来灵活调整hpreact输出分布
394 if training:
395     batch_mean = hpreact.mean(0,keepdim=True)
396     batch_std = hpreact.std(0,keepdim=True)
397     with torch.no_grad():
398         batch_mean_running.mul_(0.999).add_(0.001*batch_mean)
399         batch_std_running.mul_(0.999).add_(0.001*batch_std)
400 else:
401     batch_mean, batch_std = batch_mean_running, batch_std_running
402
403 hpreact = batch_gain*(hpreact-batch_mean)/(batch_std+1e-5)+batch_bias#这里加一个极小值，防止batch_std刚好为0
404 return hpreact
405
406
407
```

```
408 # (5) 这样强行更改hpreact矩阵分布，有没有可能导致丢失训练后学到的信息？
409 # BatchNorm 不会擦掉权重里已经学到的信息。它改变的是激活值的表示。
410 # 它确实会丢掉当前 batch 的均值和方差，但网络可以通过 gain、bias 和其他权重补偿。
411 # 实际中这个代价通常小于它带来的训练稳定收益。
412 # 但如果 batch 太小、batch 内样本不独立，或者任务强依赖绝对尺度，这种信息丢失就可能变成实际问题。
413
414
415
416 # (6) BatchNorm会导致不同样本之间存在数学上的耦合：
417 # 1.我们先观察整个样本数据，在这个只有一个hidden_layer的神经网络的流动：
418 # 一开始假设我们采用了32个样本，每个样本有3个字符作为context，每个字符用一个10维向量表示
419 # 然后1个样本embedding之后，被拼接成一个30维的向量，然后输入线性层，得到预激活值，输入激活函数，得到激活值，作为logits
420 #
421 # 2.这个过程中，每一个样本的输出值是什么，只和参数W本身有关。
422 # 3.但是在线性层和激活函数之间，引入了BatchNorm层之后，我们BatchNorm内的参数，就包含了别的样本的信息
423 # 4.所以一个样本，输入神经网络后得到的输出的logits值，就天然受到了别的样本的影响
424 # 5.所以就称为数学上的耦合
425
426
427
428 # (7) 分析这种耦合性的利弊？
429 # advantage：
430 # 1.对初始化不敏感：
431 # 没有 BN 时，W 稍微大一点，hpreact 就爆炸；稍微小一点，hpreact 就趋近 0，梯度消失。
432 # BN 把 hpreact 重新标准化，W 的尺度影响被削弱。
433 # 2.防止过饱和：
434 # 每层输入的分布被拉到零均值、单位方差附近。
435 # tanh、sigmoid 这类激活函数不再容易饱和，梯度不会轻易消失。结果是可以更大的学习率，收敛更快。
436 # 3.轻微正则化：
437 # 每个 batch 的 mean、var 都带噪声。同一个样本和不同 batch 一起训练，归一化结果不同。
438 # 这种噪声类似 Dropout，有轻微正则化效果。
439 # 4.缓解内部协变量偏移：
440 # 训练中，前面层的 W 在变，后面层收到的输入分布也在变。
441 # BN 把每层输入重新拉回稳定分布，后面层不用不停适应前面层的变化。
442 # disadvantage：
443 # 1.训练和使用不一致
444 # 训练用当前 batch 的统计量，推理用 running mean/var。两套统计量不同，模型表现可能对不上。
```

```
445 #         如果训练时 batch 小、统计量噪声大，这个差距更明显
446 #     2. 难调试：
447 #         一个样本的输出受同 batch 其他样本影响。loss 异常时，很难判断是哪个样本的问题。
448 #         同样的输入，和不同 batch 一起前向，输出不同。
449 #     3. 丢失信息：
450 #         归一化把 batch 的整体偏移和整体尺度消掉了。
451 #         虽然 gamma、beta 可以学回一部分全局量，但每个 batch 的随机波动无法恢复。如果任务需要绝对大小，BN 可能有害。
452 #     4. 样本不独立可能出问题：
453 #         如果一个 batch 里全是同一类样本，mean、var 只反映这一类，不能代表整体分布。
454 #         归一化后，样本被拉向这个有偏的中心，可能损害泛化。
455 #     5. 不适合部分模型：
456 #         RNN、Transformer 这类序列模型，每个时间步的统计量跨样本算没有意义，而且序列长度不一，batch 统计量不稳定。
457 #         在线学习场景下，来一个样本就得更新，没有 batch 可言
458
459
460
461 # (8) 我们训练时都是按照Batch_size来输入样本的。那真正使用时，往往都只输入一个样本。这时候如何消除BatchNorm层影响？
462 #     1. 会有什么样影响？
463 #         我们之前对hpreact内样本们的各个特征值，计算跨样本特征值的std和mean。
464 #         如果输入样本只有一个，那么hpreact就是一个(1,n_hidden)的矩阵，std就是0，mean就是每一个特征值本身。
465 #         然后(hpreact-hpreact.mean(0,keepdim=True))/hpreact.std(0,keepdim=True)就会退化成0/0
466
467 #     2. 第一种解法：
468 #         我们直接对整个训练集处理，计算出total_mean和total_std，然后用来替代hpreact.mean和hpreact.std。
469 #         训练时候我们采用hpreact.mean和hpreact.std，使用这个神经网络的时候，我们就用全局。
470 #         advantages：
471 #             概念简单：训练完算一次，之后固定不变。
472 #             模型使用的时候没有耦合。因为训练时的mean和std和别的样本有关。但是使用时，用的是全局mean和std，和别的样本无关
473 #         disadvantages：
474 #             需要额外遍历一遍训练集，计算量不小。
475 #             训练时用的是每个 batch 的统计量，推理时突然换成全局统计量，两者有差距。如果差距太大，推理表现会掉。
476
477
478
479 #     3. 第二种解法：
480 #         看347行代码：每一次训练，都把当前训练计算的mean和std，放一点点到全局mean
481 #         至于这样为什么可以收敛到全局mean和std。。。就是数学问题了
```

